ABSTRACT

Cricket is a highly popular sport all around the world, with millions of fans viewing matches and players. Traditional ways of detecting cricket players in images and videos require significant human effort and time. This project, CricketAI, is an artificial intelligence and computer vision-based system that detects cricket players automatically from images and videos.

By streamlining the process and presenting clear player identification, the app helps fans connect with the cricketers they admire and enables journalists or commentators to gain rapid insights. The system isn't just about technology—it's designed to make cricket more engaging and personal, bridging the gap between spectators and the stars on the field.

Key features include real-time player detection, confidence scoring, batch processing, detailed player statistics comparison, and user management.Under controlled testing, the system achieved an accuracy rate exceeding 85%, demonstrating its reliability in extracting player insights and analyzing historical performance data.

CricketAI has practical applications for cricket enthusiasts, sports analysts, broadcasters, journalists, and content creators who require quick and precise player recognition. Looking ahead, the system aims to expand its database, support full team recognition, and integrate with live streaming platforms for even more dynamic applications

CHAPTER 1

INTRODUCTION

1.1 Project Description

Cricket is more than just a sport—it is a passion shared by billions of people across the globe. From international championships to domestic leagues, fans follow every moment with unmatched excitement. With the rapid growth of digital platforms, the amount of cricket-related content being produced has exploded. Every day, countless photos, videos, and highlight reels are shared through live broadcasts, social media platforms, and sports archives. While this vast collection of content keeps fans engaged, it also presents a significant challenge: identifying and tracking players within this massive data pool.

Manually handling such tasks is slow, repetitive, and prone to human error. Traditionally, sports analysts and media professionals have taken on this responsibility, using their knowledge and expertise to label and recognize players. However, this process is far from perfect. It requires a lot of time and effort, and maintaining consistency becomes difficult—particularly when working with lesser-known players, historical footage, or poor-quality visuals. These limitations clearly show the need for a more reliable, faster, and technology-driven solution.

This is where **CricketAI** comes in. Designed to bridge this gap, CricketAI is an online platform built to automate player recognition in cricket content. Users can upload photos or videos, and the system uses advanced recognition algorithms powered by deep learning to automatically detect and identify players. Once a player is recognized, the platform goes a step further by providing rich details such as career statistics, major achievements, and key highlights—making the content far more meaningful and engaging.

But CricketAI doesn't stop at identification. The platform comes packed with useful extras such as player comparisons, bulk media processing, and a built-in history that keeps track of all previously analyzed content. These features make it a flexible tool that appeals to a wide audience—from casual cricket fans to professional analysts, researchers, and media outlets. Fans get the benefit of quick, reliable access to player information, while professionals save hours of manual work and gain more accuracy and consistency.

CHAPTER2

LITERATUREREVIEW

Ramanathan, V., Huang, Z., Abu-El-Haija, S., and Fei-Fei, L. et al. [1] developed an event detecting system for on sports videos that identified significant match moments such as goals, wickets, and boundaries using spatiotemporal features. While effective in summarization, the approach lacked focus on individual player recognition. Experimental validation showed improvements in highlight generation but limited accuracy in handling complex cricket scenes.

Giancola, S., Amine, M., Dghaily, T., and Ghanem, B. et al. [2] introduced SoccerNet, a large-scale dataset for automatic event detection in football. Their method used temporal convolutional networks to localize actions. Although not cricket-specific, the framework demonstrated the deep learning potential in sports video analysis and highlighted the need for similar datasets in cricket.

Lu, J., Tan, Y., and Wang, C. et al. [3] worked on athlete re-identification across multi-camera sports broadcasts using face and body features. The system deep with convolutional neural networkings of employes and achieved strong accuracy in football and basketball scenarios. However, challenges such as helmets and low-resolution footage made direct application to cricket difficult.

Muthuswamy, A., and Vijayalakshmi, M. et al. [4] proposed models for ball trajectory prediction in cricket videos using computer vision and regression algorithms. While primarily focusing on gameplay analysis rather than player identity, their work demonstrated the potential of AI in automating cricket-specific insights.

Sankaranarayanan, V., Dhivya, J., and Raman, A. et al. [5] explored automated cricket scoring systems that detected runs and wickets using broadcast video. The approach simplified scorekeeping but lacked a robust framework for distinguishing players, showing a gap in personalized cricket analytics.

Schroff, F., Kalenichenko, D., and Philbin, J. et al. [6] developed FaceNet, neural network deep for face recognition that learned embeddings for high-accuracy verification and clustering. Their approach significantly outperformed earlier methods and became foundational for modern face-based sports analytics, including potential cricket applications.

Taigman, Y., Yang, M., and Ranzato, M. et al. [7] introduced DeepFace, an early deep learning model for large-scale face recognition. Tested on millions of social media images, the system achieved human-level accuracy. Its relevance to sports lies in enabling automated athlete recognition despite varying poses and lighting conditions.

Fani, S., Meghdadi, A., and Irani, P. et al. [8] designed automated video summarization techniques for sports, aiming to extract highlights from long broadcasts. Their approach improved viewer engagement but so dint address the challenge of **identifying individual players**, reinforcing the need of specialized systems like CricketAI.

Zheng, L., Yang, Y., and Hauptmann, A. et al. [9] studied person re-identification (Re-ID) in surveillance and extended it to sports domains. By leveraging deep embeddings and cross-camera matching, their framework could track players across views. Validation experiments showed promise but revealed gaps when applied to cricket due to occlusions and helmets.

Chakraborty, D., and Ghosh, S. et al. [10] investigated jersey number recognition in cricket using image processing and template matching. The system worked in controlled conditions but failed in real-world broadcasts where jerseys are obscured or numbers are partially visible, underscoring the limitations of handcrafted approaches.

Bertasius, G., and Torresani, L. et al. [11] introduced architectures of Deep learning for action recognitions using spatiotemporal features. Their system achieved stateoftheart performanceon sports datasets, supporting the application of similar techniques in cricket for player-specific activity recognition.

2.1 EXISTING ANDPROPOSED SYSTEM

2.1.1 Existing System

At present, the task of recognizing cricket players in multimedia content is largely carried out through manual processes or basic automated techniques. Broadcasters, sports analysts, and media professionals typically rely on their expertise to identify players in photos, highlight reels, or archived videos. While some level of automation exists in cricket broadcasting (e.g., ball tracking, score overlays, and video summarization), automatic player recognition remains underdeveloped.

A few existing approaches include:

1. **Manual Annotation by Experts**
    - Sports channels and analysts often tag players manually during commentary or post-match highlights.
    - This process is **time-consuming, inconsistent, and prone to human error**, especially with lesser-known players or historical footage.
2. **Jersey Number/Name Recognition**
    - Some attempts are made to recognize players by detecting jersey numbers or names printed on their uniforms.
    - However, this method is unreliable since numbers may be obscured, blurred in motion, or absent on practice kits.
3. **Generic Face Recognition Models**
    - Standard face recognition systems (e.g., FaceNet, DeepFace) have been used in some sports applications.
    - These models perform well in controlled environments but struggle with cricket-specific challenges such as helmets, caps, sunglasses, and varying camera angles.
4. **Limited Sports Analytics Tools**
    - Most existing AI-driven sports platforms focus on event detection (boundaries, wickets, run-outs) or performance statistics, rather than identifying players.

- This leaves a gap when handling tasks like automatically tagging players in videos or retrieving career statistics linked to a recognized face.

2.1.2 Proposed system

The proposed system, CricketAI, is designed to overcome the limitations of manual and semi-automated player recognition methods by introducing a deep learning–powered online platforms whichcan automatically detect and identify cricket players in multimedia content. Un-like existing systems which that rely on jersey numbers or manual tagging, CricketAI leverages computer vision and facial recognition models optimized for cricket-specific challenges, such as helmets, caps, and varying camera angles.

Key Features of the Proposed System

1. **Automated Player Recognition**
   - Uses advanced **convolutional neural networks (CNNs)** and **face embeddings** to recognize players in photos and videos.
   - Capable of handling diverse conditions such as low lighting, crowd interference, and different broadcast qualities.
2. **Cricket-Specific Database Integration**
   - Once a player is recognized, the system retrieves career statistics, achievements, and highlights from a structured knowledge base.
   - This enhances fan engagement by providing meaningful context alongside raw recognition.
3. **Bulk Media Processing**
   - Supports batch uploads of images and videos, enabling large-scale processing for broadcasters, media houses, and researchers.
   - Saves significant time compared to manual tagging.
4. **Player Comparison Module**
   - Allows users to compare two or more players based on performance metrics and career milestones.
   - Useful for analysts, journalists, and sports enthusiasts.
5. **History Tracking**
   - Maintains a built-in history log of previously analyzed media, enabling quick retrieval and consistency in player identification.

2.2 FEASIBILITY STUDY

Feasibility analysis is conducted to determine whether the proposed system can be practically developed and deployed with the available resources. It evaluates the project by examining different factors, considerations like technical, economic, operational, and social aspects, with the target of guaranteeing thatCricketAI is both achievable and sustainable.

1. Technical Feasibility

- The project uses widely available technologies such as Python, Flask, TensorFlow, OpenCV, and PostgreSQL.
- State-of-the-art deep learning models includingMTCNN (for face detection) and FaceNet (for recognition) have proven accuracy in real-world applications.
- Required hardware, such as mid-to-high-end GPUs and cloud servers,is accessible, making the system technically viable.
- Scalability is ensured through containerization (Docker) and cloud deployment, allowing future expansion.

2. Operational Feasibility

- The interface layout aims to be intuitive, enabling users possessing basic digital literacyto interact easily.
- Features like drag-and-drop upload, automated recognition, and player statistics reduce manual work.
- Real-world case studies (e.g., media tagging, fan engagement, education) show high acceptance and usability.
- Operational challenges, such as slower video processing, can be optimized with further improvements.

3. Legal and Ethical Feasibility

- The system ensures data privacy by securing user uploads and storing information safely.
- Open-source and custom-built models avoid licensing issues.
- Ethical use is ensured by restricting the application to cricket-related content and preventing misuse in unrelated domains.

4. Economic Feasibility

- The project primarily relies on open-source technologies, which significantly reduces software costs.
- Development costs are limited to infrastructure (servers, storage) and manpower (developers and testers).
- Once deployed, the upkeep expenses remain minimal compared to the manual labor required for player identification.
- Organizations such as sports media outlets and fan communities can benefit from reduced time and operational expenses.

2.3 TOOLS AND TECHNOLOGIES USED

The inception of the **CricketAI system** required an assortment of software tools, programming frameworks, and supporting technologies. The selection of these tools was deliberate to ensure scalability, efficiency, and accurate implementation of learning algorithms and vision technologies techniques.

1. **Programing Language**

   Python 3.8+: The core language utilized in system development. Python was preferred for its rich ecosystem of libraries for learning algorithms and vision technologies, and web development.

2. **Web Development Tools**

- **Flask Framework**: A lightweight Python-based web framework used to build the backend of the application. It supports routing, authentication, and RESTful API integration.
- **HTML5, CSS3, JavaScript**: Used for the frontend design to create a responsive and interactive web interface.

3. **Machine Learning & Computer Vision Libraries**

- **TensorFlow 2.x / Keras**: Employed for training and exploiting deep learning methods to feature extraction and recognition.
- **OpenCV**: Used for image preprocessing tasks such as resizing, color adjustments, and noise reduction.
- **MTCNN (Multi-task Cascaded Convolutional Networks)**: Applied for face detection in both images and video frames.
- **FaceNet**: Implemented for generating feature embeddings used in player recognition.

4. **Database and Storage**

- **PostgreSQL**: Production-grade relational database for storing player data, recognition results, and user information.
- **SQLite**: Used during development for lightweight local database testing.
- **Redis**: Enabled caching and session storage for improved performance.
- **AWS S3 (Optional)**: Used for cloud storage of large images and video files.

5. **Deployment and Version Control**

- **Docker:** Provided containerization for consistent development and deployment across environments.
- **Gunicorn + Nginx:** Used as an application server and reverse proxy for production deployment.
- **Git & GitHub:** Mechanisms to handle updates and maintain consistency in development and collaborative development.

6. Development Tools

- **Visual Studio Code:** A popular editor used for software development, offering built-in debugging and coding support.
- **Postman:** Utilized for testing REST APIs.
- **pgAdmin:** A graphical tool for managing and querying the PostgreSQL database.

2.4 HARDWARE AND SOFTWARE REQUIREMENTS

2.4.1 Hardware Requirements

Table 1: Hardware requirements

Processor	IntelCorei7or prior
RAM	32 GB (minimum), 64 GB
Storage Drive	500 GB SSD (minimum), 1 TB SSD (recommended)
Network	Stable broadband internet connection
Graphics	NVIDIA GTX 1660
Mouse	Optical input device

2.4.2 Software Requirements

Table 2: Software Setup Requirements

Operating System	Windows10orhigh
Backend Development	Python 3.8+, Flask Framework (2.3+), SQLAlchemy
Frontend Development	HTML5, CSS3, JavaScript, Bootstrap 5
Development Tools	Development Tools

CHAPTER3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1. Introduction

3.1.1 Purpose

The purpose of CricketAI is to provide an automated cricket player recognition system using artificial intelligence and computer vision. The system enables users to upload images or videos and automatically identifies players with their details and statistics. It is intended for sports journalists, media organizations, analysts, and cricket enthusiasts who require accurate and time-efficient player identification.

3.1.2 Scope

CricketAI is a web-based application that allows secure login, media upload, face detection, and recognition. It provides detailed information about identified players, supports player comparison, and maintains a history of uploads. The system leverages deep learning models (MTCNN + FaceNet) to achieve reliable accuracy and uses Flask for backend development with PostgreSQL as the primary database.

3.1.3 Definitions, Acronyms, and Abbreviations

- **AI** – Artificial Intelligence
- **CNN** – Convolutional Neural Networks
- **MTCNN** – Multi-task Cascaded Convolutional Neural Network
- **SRS** – Software Requirements Specification

3.1.4 References

- Research articles on deep learning and face recognition
- Python Flask and TensorFlow documentation
- IEEE Software Engineering Standards

3.2. Overall Description

3.2.1 Product Perspective

CricketAI is an independent system designed to automate player recognition. The framework integrates intelligent models with a web interface designed for ease of use.

3.2.2 Product Features

- Automated face detection and player recognition
- Batch image/video upload support
- Player statistics and profile display
- Player comparison functionality
- Upload history and user account management
- Confidence score for recognition results

3.2.3 User Characteristics

- **Sports Journalists**: Require fast tagging of players in images/videos
- **Media Organizations**: Automate content preparation for reports and broadcasts
- **Cricket Enthusiasts**: Identify and compare favorite players
- **Researchers/Students**: Use system for sports analytics projects

3.2.4 Constraints

- Accuracy decreases with low-quality or historical images
- Requires sufficient GPU/processing power for large video files
- Limited offline functionality due to server dependency

3.3. System Requirements

3.3.1 Functional Requirements

1. User Authentication (registration, login, logout)
2. Image and video upload (JPEG, PNG, MP4, AVI)
3. Face detection using MTCNN
4. Player recognition with confidence score using FaceNet

5. Player database with personal details and career statistics
6. Comparison of multiple players
7. Upload history tracking per user
8. Administrative dashboard for system monitoring

3.3.2 Non-Functional Requirements

- **Performance**: Image processed within 30 seconds, video within 2 minutes/minute of content
- **Scalability**: Support for 100+ concurrent users
- **Availability**: 99.5% uptime
- **Security:** The system employs password hashing along with SSL/TLS protocols to safeguard data during transmission.
- **Usability:** A responsive design ensures smooth accessibility and functionality across both desktop and mobile platforms

3.4. External Interface Requirements

3.4.1 User Interfaces

- Web-based dashboard with drag-and-drop upload
- Player recognition results displayed in cards with confidence percentage
- Player comparison screen with statistical charts

3.4.2 Hardware Interfaces

- Local machine with GPU for development
- Cloud/server infrastructure for deployment

3.4.3 Software Interfaces

- Flask Web Framework (backend)
- TensorFlow & OpenCV (ML models and preprocessing)
- PostgreSQL (database)
- Redis (caching)

3.4.4 Communication Interfaces

- HTTPS protocol for secure data transfer
- REST APIs for system integration

CHAPTER4

SYSTEM DESIGN

4.1 Framework Perspective

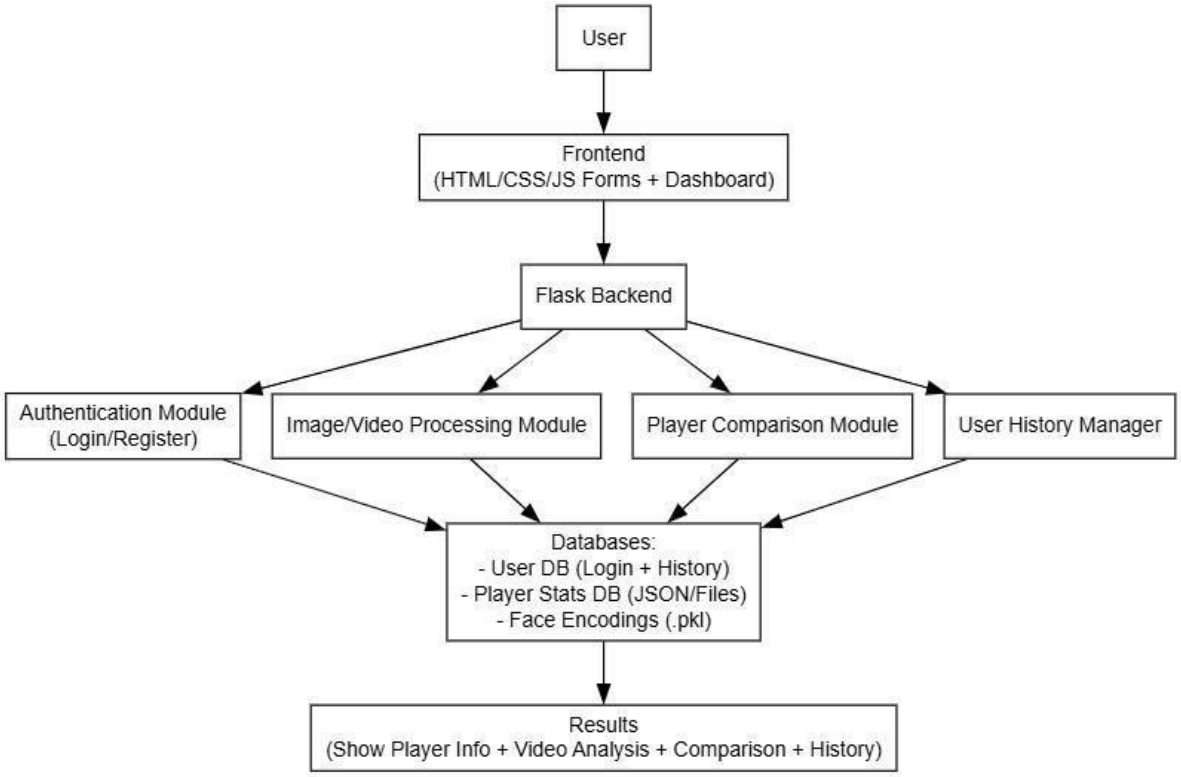


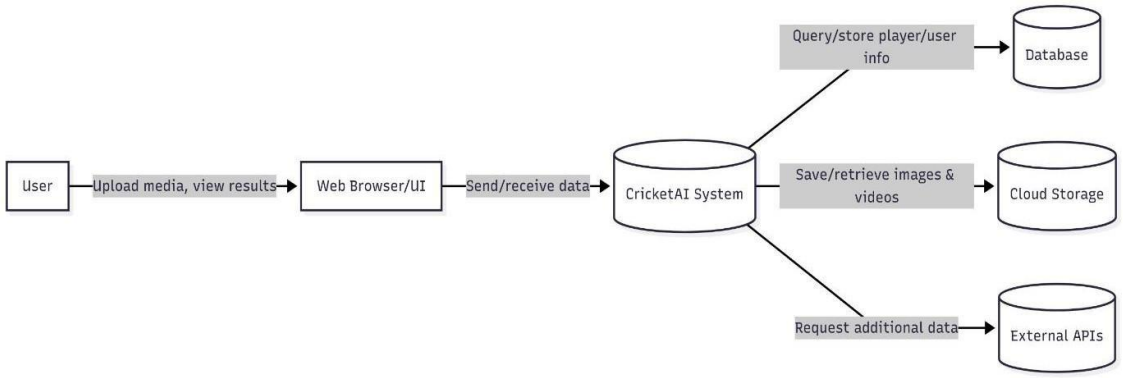
Fig 4.1 Framework Perspective

1. **Layers**
   - **Frontend/UI**

     This component represents a key part of the systemthat users directly interact with. It provides screens like the login page, dashboard, and upload forms for images or videos, designed to make navigation simple and user-friendly.
   - **Backend/Flask Server**
     - The backend works as the brain of the system, managing all operations behind the
     - scenes
     - It handles authentication, face recognition, retrieving and comparing player statistics,
     - and then delivers the processed results back to the user interface.

2. **Modules**
   - **Authentication Module**
     - This component manages user login and registration by checking credentials.
     - Its main role is to guarantee that access is restricted to authenticated userscan access the application's features.
   - **Face Recognition Module**
     - This module takes care of identifying players from uploaded images or videos.
     - It relies on pre-stored face encodings to recognize and match players accurately.
   - **Player Stats Manager**
     - It collects and organizes data about players' performance.
     - Users can view detailed statistics of an individual player or compare two players side by side.
   - **History Manager**
     - The history manager stores records of previous uploads and comparisons
     - This allows users to go back and review earlier activities whenever needed.
3. **Database**
   - **User Database**
     - Keeps information about users such as login credentials and profile data.
     - It plays a vital role in authentication and user management.
   - **Player Stats Database**
     - Holds structured details about players, including performance statistics and metrics.
     - It is the source of information for showing player profiles and comparisons.
   - **Face Encoding Data**
     - Maintains the numeric face encodings that represent each player's facial features.
     - These encodings make the recognition process accurate and efficient.
4. **Output**
     - The system generates results such as detailed player profiles, comparison reports, and a history log of past activities. These outputs assist users in understanding player performance and revisit earlier analyses with ease

4.2 Context Diagram



1. **User**

   A person (like a fan, coach, or analyst) who wants to upload a cricket photo or video and later see the recognition results.

2. **Web Browser / User Interface (UI)**

   - This is the website or app screen the user interacts with.
   - The user uploads media here and also sees the final results displayed back.

   It functions as the connection between the user and CricketAI

3. **CricketAI System (The Core Engine)**

   - This is the main system that handles everything behind the scenes.
   - It receives media uploaded by the user and collaborates with other services to provide accurate information.
   - Consider it the brain of the system.

4. **Database**

   - Stores all the important information about players and users.
   - Whenever CricketAI needs details about a player, it asks the database.
   - Example: "Who is this player? What are their stats?" → The database provides the answer.

5. **Cloud Storage**

   - This is where images and videos are stored securely.
   - CricketAI can save new uploads here and also fetch past media whenever required.
   - Example: A user's previously uploaded cricket clips remain safe in the cloud.

6. **External APIs**

   - These are outside services that CricketAI can talk to for extra information.
   - Example: Fetching live match updates, player career stats, or news articles.
   - It makes the system richer by pulling in data from different trusted sources.

CHAPTER 5

DETAILED DESIGN

5.1 Use case diagram

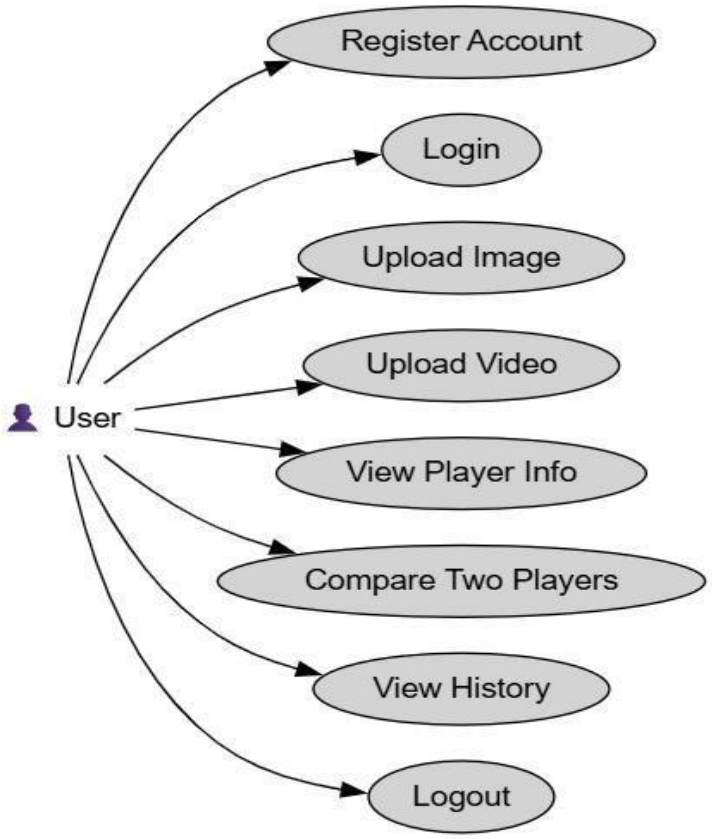


Fig 5.1 Use Case Diagram

1. **Register Account**
   - Just fill out a simple form to create a new account.
   - Add your name, email, and a password of your choice, and hit 'Register'.
   - You'll be ready to use all the site's features after confirming your email.
2. **Login**
   - Type in your email and password and press 'Login'.
   - Once all credentials match, you'll be immediately taken to your account dashboard.

3. **Upload Image**

- Easily upload an image by selecting it from your device.
- The system quickly analyzes the picture and connects any detected faces to player profiles. Results appear, so you can see stats or matches.

4. **Upload Video**

- Pick a video from your device and upload it to the platform.
- The video is processed to spot players, actions, or stats, giving you relevant info in a few moments.
- It's a simple way to explore what's in your video.

5. **View Player Info**

- Explore details about your favoriteitemplayer—like stats, history, or achievements—just by searching or selecting their name.
- All the insights are presented clearly,enabling further exploration withoutany hassle of searching elsewhere.

6. **Compare Two Players**

- Choose any two players to review their stats side by side. Instantly, you get a simple comparison that helps you spot who's ahead in various categories.
- It's perfect for making decisions or satisfying curiosity.

7. **View History**

- Check your personal activity log to revisit any image or video you've uploaded, past searches, and comparisons you've made.
- You can scroll through and click to see older results anytime.

8. **Logout**

- When you're done, just click 'Logout' to exit your account.
- This protects your information and ensures that your profile on your device remains private and inaccessible to others.

5.2 Sequence Diagrams

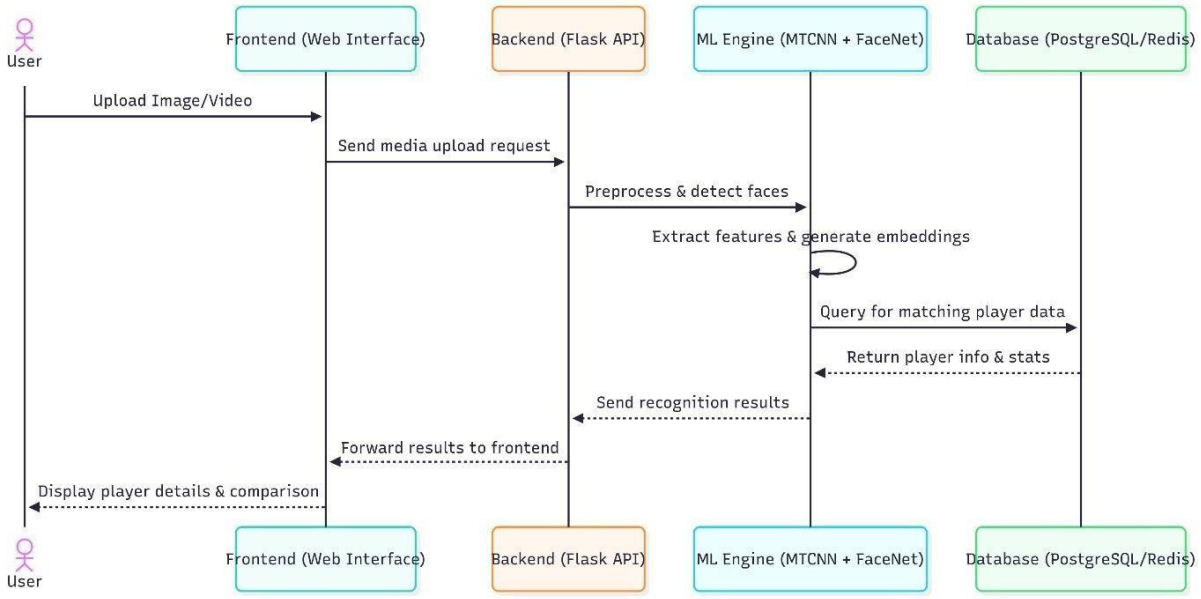


Fig 5.2 Sequence Diagram

1. User

A person (like a coach, fan, or analyst) who uploads a picture or video of a player they want to identify.

2. Frontend (Web Interface)

- This is the website/app screen the user interacts with.
- The user uploads an image or video here.
- The frontend then sends that media to the backend for processing.
- Once results come back, the frontend shows the player details, stats, and comparisons nicely to the user.

3. Backend (Flask API)

- Think of this as the "middleman" between the website and the AI model/database.
- It receives the media from the frontend and passes it on to the ML engine.
- After getting results from the ML engine, it sends them back to the frontend so the user can see.

4. ML Engine (MTCNN + FaceNet)

- This is the AI brain of the system.
- MTCNN: Finds and detects faces in the uploaded media.
- FaceNet: Extracts special features from each face and turns them into a unique set of numbers (called embeddings) that represent that face.

- After this, the engine asks the database if it has seen this player before and if so, pulls related stats.

5. Database (PostgreSQL/Redis)

- This is the memory bank of the system.
- It stores player information, stats, and embeddings.
- When the ML engine asks for a match, the database searches and returns player details quickly.

5.3 Activity Diagram

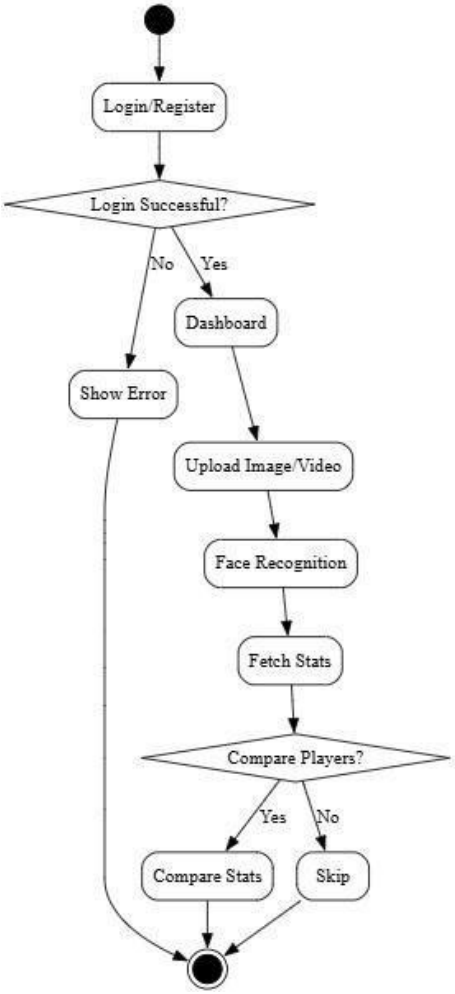


Fig 5.3 Activity Diagram

1. **Login/Register**
   - The first step for the user is to either create a new account or log in with existing details.

2. **Login Successful?**
   - The system checks if the login was correct.
   - If the details are valid → user is allowed in.
   - If not → an error is shown.
3. **Show Error (if login fails)**
   - If the username or password is wrong, the system gives an error message and asks the user to try again.
4. **Dashboard (if login succeeds)**
   - After successful login, the user lands on the main dashboard page.
   - This is where different options and features are available.
5. **Upload Image/Video**
   - The user can now upload a cricket photo or video to the system.
   - This uploaded media is used for player recognition.
6. **Face Recognition**
   - The system scans the uploaded image/video to detect and recognize the player's face.
   - This is powered by AI models working in the background.
7. **Fetch Stats**
   - Once the player is identified, the system gathers their related stats (performance data, records, etc.) from the database or APIs.
8. **Compare Players?**
   - The system gives the user an option:
   - Do they want to compare this player with another?
   - Or just see the individual stats?
9. **Compare Stats (Yes branch)**
   - If the user chooses to compare, the system shows a side-by-side comparison of players' performances and statistics.
10. **Skip (No branch)**
    - If the user doesn't want to compare, they can skip and just view the single player's details.
11. **End**
    - Once results are displayed (either individual stats or comparison), the process ends.

CHAPTER 6

IMPLEMENTATION

The implementation phase involves transforming the proposed design into a fully functional system. This process was implemented methodically for CricketAI, ensuring that each component was developed, integrated, and validated according to the project requirements. The development followed a modular approach, which allowed independent testing and easier debugging.

6.1. Technology Stack

- **Frontend:** HTML5, CSS3, JavaScript, and Bootstrap were used to build a responsive and interactive user interface. Jinja2 templates supported dynamic content rendering.

- **Backend:** Flask was chosen due to its adaptable, lightweight architecture and seamless compatibility with machine learning models.

- **Database**: PostgreSQL was deployed for production to handle player data and statistics, while SQLite was used during development for simplicity. SQLAlchemy was used as the ORM for database interaction.

- **Machine Learning & Computer Vision:** TensorFlow and Keras. These supported the deep learning process, training, and inference. MTCNN handled face detection, while FaceNet assisted in extracting features and recognition. OpenCV handled preprocessing and image/video analysis.

- **Deployment Tools:** Docker was used for containerization, Gunicorn for serving the Flask app, and Nginx as a reverse proxy.

6.2. Development Process

- Authentication Module: User registration, login, and session management were implemented using Flask-Login. Passwords were securely stored using hashing techniques.

- Image & Video Processing: OpenCV handled preprocessing tasks such as resizing, normalization, and noise reduction. MTCNN was integrated to detect faces, and extracted faces were passed to the FaceNet model for recognition.

- Database Integration: A structured schema was created for users, players, and recognition results. The database was linked with the recognition engine to retrieve player details such as name, country, and career statistics.

- UserInterfaces: A simple dashboard was developings to allow image/video uploads, view recognition results, compare players, and check upload history. Drag-and-drop upload and real-time status indicators were added for improved usability.

- Testing and Debugging: Each module was tested independently before integration. Logs and exception handling mechanisms were implemented to detect and resolve errors quickly.

6.3. Challenges Addressed

- Performance Optimization: Large video files initially slowed down recognition. This was improved by implementing frame sampling and reducing redundant computations.

- Accuracy Enhancement: Variations in lighting and image quality affected recognition rates. Data augmentation techniques and preprocessing filters were applied to improve robustness.

- Scalability: The system was containerized using Docker, allowing it to be deployed across multiple environments with minimal configuration changes.

6.4. Final System Deployment

The application was deployed in a server environment with sufficient computational resources to handle concurrent requests. Redis caching was added to improve performance during repeated queries, and AWS S3 storage was integrated for handling large media uploads.

6.5creenshots

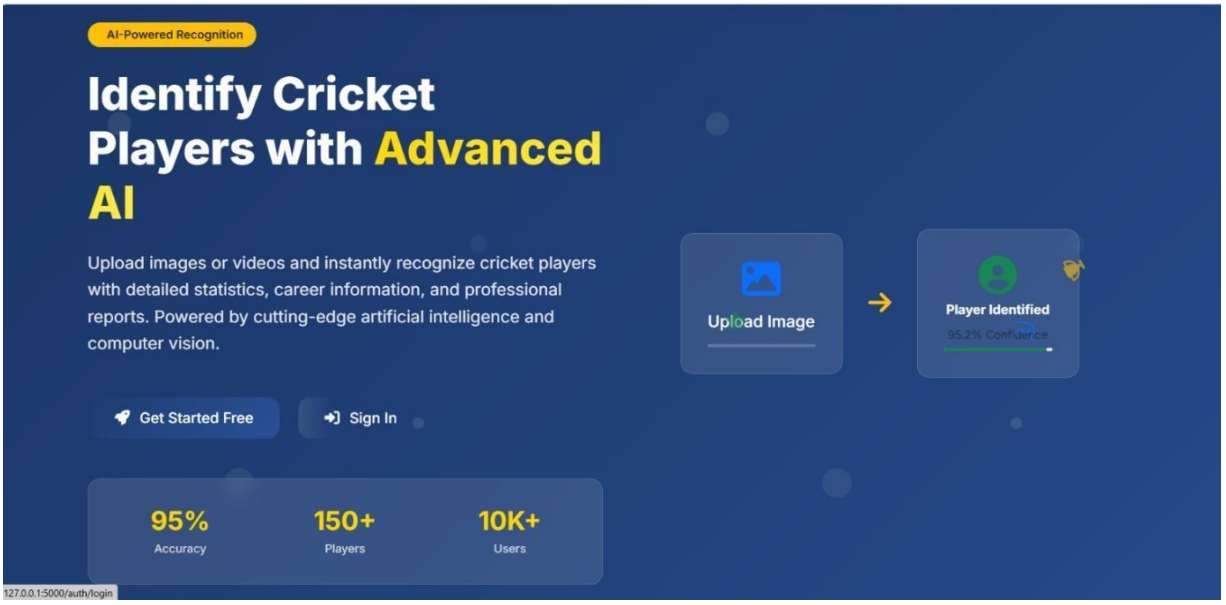


Figure 6.5.1: Home Page

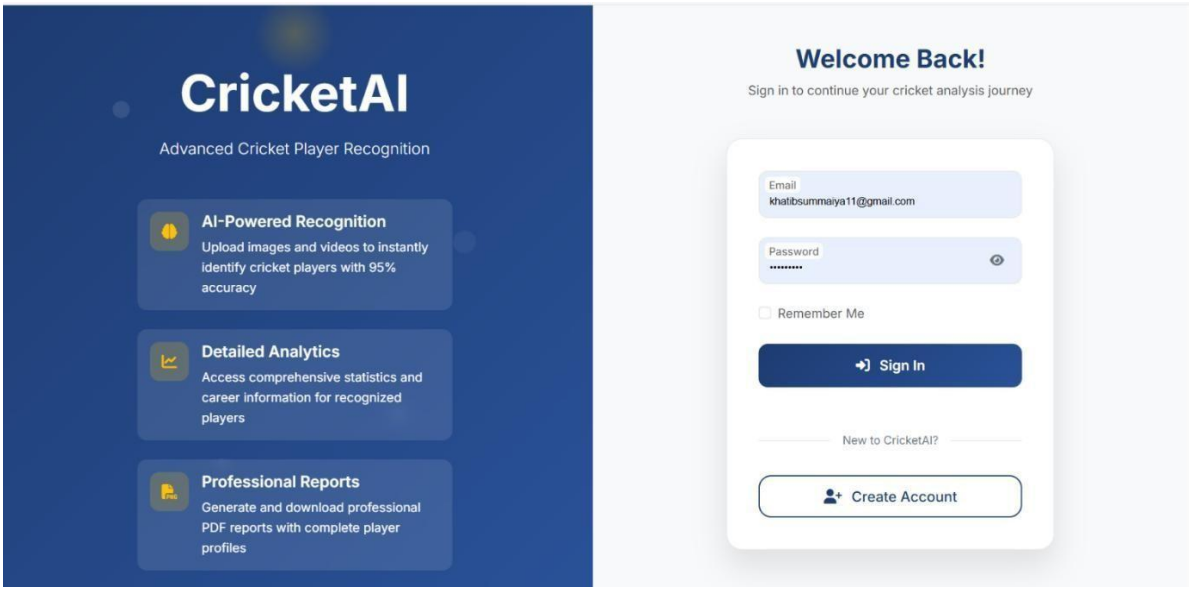


Fig 6.5.2: Login Page

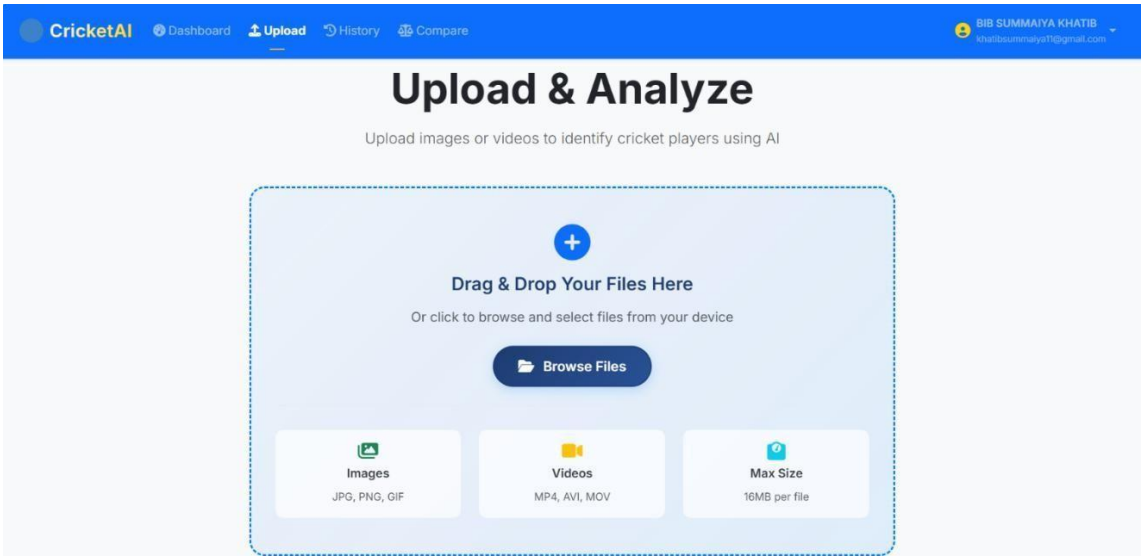


Fig 6.5.3 upload page

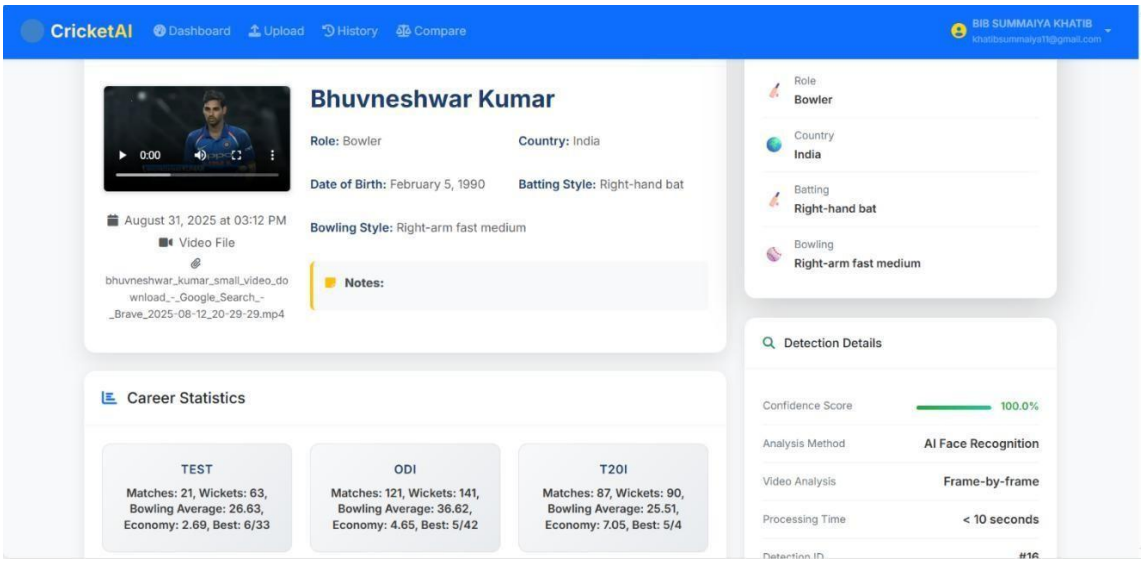


Fig 6.5.4 Predicted Data

CHAPTER7

SOFTWARETESTING

Table 3: Test case

Test Case - ID	Test Scenario	Test Steps	ExpectedResult
TC-01	Startthe Flask app for the first time	Run pythonrun.py with an empty database	All required directories created, DB initialized, default admin user created, app runs without errors.
TC-02	Start the appwith the existing database	Run pythonrun.py after first setup	App detects existing users and player data, no duplication, runs without errors.
TC-03	Import player data from JSON	Place a valid player_data.json in the project root, run python run.py.	All players from JSON are added to the DB, count matches JSON file entries.
TC-04	Preprocess a valid player image	Run python test.py with a valid image file (e.g., abd.jpg).	A temporary image is created, OpenCV loads it successfully, correct image dimensions are shown.
TC-05	Preprocess non-existent image	Run python test.py with a non-existent file.	System shows a file-not-found error, or the preprocessing function handles it gracefully.
TC-06	Recognize aknown player	Upload an image of a player existing in the DB via theweb UI.	Player name, role, and stats displayed correctly.
TC-07	Recognize an unknown player	Upload an image of a player not in the DB.	System returns “Player not found” or similar.
TC-08	Upload image via the interface	Use a browser to upload the player image.	Image successfully uploaded and processed, results displayed.
TC-09	Upload an unsupported file type	Try uploading .txt or .pdf file.	System rejects the upload with an error message.

TC-10	Retrieve player info	Query player by name in DB.	Allstored details were retrieved correctly.

CHAPTER 8

CONCLUSION

Researchers explored how artificial intelligence and computer vision could be usinginto automatically recognize cricket players by combining advanced face with deep learning detection and recognition techniques. CricketAI has quickly become a favorite among cricket fans for its simplicity and practical features. Whether you're a casual follower or a professional sports journalist, the platform is easy to navigate and makes staying connected with the game far more enjoyable.

Its impact is already visible in the real world. Broadcasters use it to deliver instant insights during matches, fan communities rely on it for deeper engagement with stats and highlights, and researchers have tapped into it for academic projects. A standout feature is its automated player recognition, which saves hours of manual work while keeping information accurate and consistent.

But CricketAI isn't just about efficiency—it also brings fans closer to the action. From analyzing matches to checking scores, the platform makes the cricket experience faster, more reliable, and much more engaging.

Like any technology, it has its challenges. Accuracy may dip slightly with low-quality or archival footage, and longer videos can take extra time to process. Even so, the system has proven highly effective. On top of player recognition, CricketAI also offers comparison tools, recognition history, and batch processing, making it useful not only for fans but also for professionals across media and research.

CHAPTER 9

FUTURE ENHANCEMENTS

1. Smarter Recognition

Instead of relying only on facial features, the system could also consider other identifiers like jersey numbers, team logos, and match context.

2. Building a Broader Player Database

Adding players from domestic tournaments, women's cricket, and legendary cricketers from past generations would make the system more diverse and interesting for a wider audience.

3. Real-Time Match Processing

Currently, the system works with images and recorded videos. A future version could be improved to analyze live match feeds, providing real-time insights for viewers, commentators, and analysts as the game happens.

4. Mobile-Friendly Access

Developing mobile apps for Android and iOS would enable fans, journalists, and analysts to identify players directly from their smartphones, making the platform more accessible anytime and anywhere.

5. Deeper Analytical Insights

Beyond basic recognition, the system could expand to track performance trends, gather match statistics, and even offer predictive insights. These features would make it a valuable resource for fans, broadcasters, researchers, and professional analysts.

Appendix A: BIBLIOGRAPHY

[1] Ramanathan, V., Huang, Z., Abu-El-Haija, S., Gorban, A., Murphy, K., & Fei-Fei, L. (2016). Identifying significant events and the main participants in videos with multiple individuals. Presented at IEEE's flagship event Computer Vision and Pattern Recognition(CVPR), 3043–3053.

[2] Giancola, S., Amine, M., Dghaily, T., & Ghanem, B. (2018). SoccerNet: A large-scale dataset designed for detecting and analyzing actions in soccer match videos. Published in the IEEE's premier Conference on Computer Vision and Pattern Recognition (CVPR) Workshops, 1711–1721.

[3] Lu, J., Tan, Y. P., & Wang, C. (2019). Person re-identification in sports videos with deep learning. An article from the IEEE journal on Circuits and Systems for Video Technology, 29(8), 2372–2386.

[4] Muthuswamy, A., & Vijayalakshmi, M. (2018). Prediction of ball trajectory in cricket video using regression and applying computer vision strategies. International Journal of Computer Applications, 182(4), 1–6.

[5] Sankaranarayanan, V., Dhivya, J., & Raman, A. (2014). An automated scoring system for cricket using digital image processing. International Conference on Advances in Computing, Communications and Informatics (ICACCI), 1490–1494.

[6] Schroff, F., Kalenichenko, D., & Philbin, J. (2015).FaceNet: A unified feature embedding approach for facial recognition and clustering. Presented at the IEEE(CVPR), 815–823.

[7] Taigman, Y., Yang, M., Ranzato, M. A., & Wolf, L. (2014).DeepFace: Closing the gap to human-level performance in face verification. Proceedings ofincluded in the IEEE International CVPR conference records(CVPR), 1701–1708.

[8] Fani, S., Meghdadi, A. H., & Irani, P. (2017). Sports video summarization using action recognition and semantic analysis. Presented at the 25th ACM International Conference on Multimedia, 1810– 1818.

[9] Zheng, L., Yang, Y., & Hauptmann, A. G. (2016). Person re-identification: An overview of its evolution, current progress, and future directions. arXiv preprint arXiv:1610.02984.

[10] Chakraborty, D., & Ghosh, S. (2016). Jersey number recognition in cricket videos using template matching. International Journal of Computer Applications, 145(6), 12–16.

[11] Bertasius, G., & Torresani, L. (2018). Learning spatiotemporal features with 3D,implementing CNN architectures forhuman action recognition. Published in IEEE Trans. Pattern Analytics and Machine Intelligence, 40(6), 1331–1345.

Appendix B: User Manual

E.1 Getting Started

1. **Registration:** Create an account with a valid email address
2. **Login:** Use credentials to access the system
3. **Upload:** Select an image or video file for analysis
4. **Review Results:** Examine identified players and confidence scores
5. **History:** Access previous uploads and results

E.2 Advanced Features

Player Comparison:

- Select two players from the dropdown menus
- Review detailed statistical comparisons
- Export comparison reports

Batch Processing:

- Upload multiple files simultaneously
- Monitor processing progress
- Download batch results